case: each retailer can have their own system with which we need to interact:

- They can sell items offline, for instance: if that happens, our info about their stock can become stale. Similarly, we can get off-sync in case a shop gets items re-stocked.
- In listing 11.2 and figure 11.3 we assume that a call to `shop.order` will succeed, but since it's likely to be a remote call over HTTP, there are many reasons why it could fail independently on the item availability in the shop's inventory: the call could timeout, their application could crash and be unreachable, etc… If we don't check their response, we will never know if the order was successfully placed. And if we do check, but never get a response, what should we do?

These are extremely challenging issues, that we'll try to solve in the next sections.

11.2 Centralized Application

Let's keep architectural questions aside for the moment, assume we handle in-house (within the same virtual machine running our web application) all orders for retailers and focus on the first round of questions: at some point we asked how should we choose which shop or combination of shops is the best to serve a customer. Neither k-d trees, S-trees nor any other data structure can (nor should) answer this question: this is a business-related decision that changes from